

SERVICE ACTUEL · FILES

Fichiers, dossiers et traitements documentaires

8 capacités analysées

services/files · Python, FastAPI, SQLAlchemy asynchrone ; PostgreSQL pour hiérarchie et index, S3/MinIO pour objets, ARQ et Redis/Valkey pour jobs ; Kafka pour faits et antivirus/extraction spécialisés.

Fournir un espace de fichiers personnels ou de projet, navigation, recherche, copies, téléchargement, corbeille et traitements asynchrones. Ce service n'est pas aujourd'hui une autorité d'édition FILESET.

Lecture de code

Pas de qualification runtime nouvelle

Domaines cible : Files, Writer, Publisher, Audit, Maintenance, IAM, Catalog, Semantics, Embeddings.

Périmètre de lecture

- AGENTS.md ; rapport Files existant ; routes files, folders, projects, search, trash, jobs et config_assets.
- ContentStore, parties ciblées des services de fichiers, chunks, assets, extraction, worker et intention de suppression.

Objets et cycle de vie

- File et Folder portent identité, hiérarchie et état logique en PostgreSQL. Les octets sont des objets S3 à clés plates.
- Le SHA-256 actuel décrit le contenu ; il ne fournit ni édition publiée ni déduplication globale prouvée.
- ConfigAsset est séparé des fichiers privés ; les jobs et intentions de suppression ont leurs propres états.

Dépendances du service

- IAM et APISIX contrôlent ressources de chemin et destinations de corps.
- PostgreSQL, S3/MinIO, ARQ avec Redis/Valkey, antivirus et extraction PDF.
- Kafka relaie les faits actuels ; sa substitution fait partie de la cible sans JVM.

Fonctionnalités et compatibilité à préserver

FS-01 Cible non qualifiée

Recevoir un fichier et contrôler sa destination avant stockage

COMPORTEMENT OBSERVÉ

POST upload valide la destination via IAM lorsque la gateway ne peut pas la déduire du chemin. Pour un dossier neuf, le handler peut réessayer le même contrôle pendant la propagation de son tuple, sans substituer un droit de projet. ContentStore borne la taille, détecte le MIME dans les octets, applique la politique et calcule SHA-256 avant l'écriture S3. L'objet utilise une clé UUID/nom ; le hash stocké ne constitue pas une déduplication globale. La ligne initiale peut attendre son scan antivirus.

UTILISATEURS ET RÔLES

Utilisateur ou intégration qui ajoute un fichier.

INTERFACES DANS LE CODE

POST /api/v1/files/upload ; `_authorize_upload_target` ; `ContentStore.ingest_streaming`.

DONNÉES ET ÉTATS

File PostgreSQL, clé de blob, taille, MIME détecté, SHA-256 et état de scan.

DÉPENDANCES ET ÉCHANGES

IAM broker, gateway, stockage S3 et worker antivirus.

COMPORTEMENT CIBLE PROPOSÉ

Files Rust crée une session d'upload et écrit dans un espace privé durable. Le reçu de réception ne vaut pas publication. Writer prépare l'index FILESET ; Publisher approuve l'édition Iceberg après audit, avec origin.* au candidat.

MIGRATION DE CETTE CAPACITÉ

Préserver IDs, noms, limites et erreurs. Auditer les objets existants puis adopter uniquement une version immuable avec hash contrôlé ; reconstruire une édition publiée sans ériger la navigation PostgreSQL en checkpoint.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un MIME falsifié ou un fichier trop grand n'atteint pas le stockage. Un dossier refusé ne devient pas autorisé par son projet. Aucun octet reçu n'est lisible avant son édition approuvée.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/api/v1/files.py:63](#) Contrôle de la destination et propagation des droits.

[monorepo](#) · [services/files/src/file_management/storage/content_store.py:170](#) Validation complète avant écriture des octets.

FS-02 **Cible non qualifiée****Assembler les uploads par morceaux et nettoyer les abandons****COMPORTEMENT OBSERVÉ**

L'upload par chunks utilise propriétaire, projet, nom et `upload_id` pour isoler le fichier temporaire. Le nom est ramené à son basename ; l'écriture prend flock et revérifie la taille sous verrou. Le service calcule ensuite le checksum du fichier assemblé et poursuit le parcours d'upload. Le worker nettoie les uploads périmés. Ce mécanisme dépend du fichier local et ne démontre pas une reprise depuis un worker arbitraire.

UTILISATEURS ET RÔLES

Clients qui transfèrent des fichiers volumineux.

INTERFACES DANS LE CODE

POST /api/v1/files/upload/chunk ; `ChunkedUploadService.upload_chunk` ; `cleanup_stale_uploads`.

DONNÉES ET ÉTATS

Fichiers temporaires locaux et métadonnées de requête ; objet final et ligne File après assemblage.

DÉPENDANCES ET ÉCHANGES

Disque temporaire du service, stockage S3 et ARQ.

COMPOTEMENT CIBLE PROPOSÉ

Sessions Rust durables avec identifiant de transfert, plages ou parties, taille attendue, checksum et état de finalisation. Les workers reprennent depuis le stockage durable ; la fermeture du transfert produit un candidat FILESET puis une publication WAP.

MIGRATION DE CETTE CAPACITÉ

Garder la route compatible mais router chaque session vers le protocole durable. Terminer ou invalider explicitement les sessions locales ouvertes avant de retirer les anciens pods.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Interrompre une réception puis reprendre sur un autre worker restitue exactement le même fichier. Le rejeu d'une partie ne la double pas. Un nettoyage n'efface pas un upload actif.

Références de l'analyse

monorepo · [services/files/src/file_management/services/chunked_upload_service.py:85](#) Isolation des chemins et noms.

monorepo · [services/files/src/file_management/services/chunked_upload_service.py:105](#) Verrou et contrôle de taille.

monorepo · [services/files/src/file_management/workers/tasks.py:1127](#) Nettoyage des transferts abandonnés.

FS-03 **Cible non qualifiée****Naviguer, déplacer et copier les fichiers et dossiers****COMPOTEMENT OBSERVÉ**

Le service expose contenu personnel, contenu de projet, résolution de chemin, dossiers simples ou par lot, renommage, déplacement et copie. La hiérarchie réside en PostgreSQL ; les clés S3 sont plates. Les destinations fournies dans le corps demandent une autorisation IAM. Les copies de dossiers peuvent produire un job ; son statut est réservé à l'utilisateur qui l'a demandé. Un non-propriétaire voit 404. Si un statut Redis manque après acquittement de l'intention durable, l'API retourne 503 plutôt qu'un pending permanent.

UTILISATEURS ET RÔLES

Contributeurs de projets et propriétaires d'espace personnel.

INTERFACES DANS LE CODE

/api/v1/my-files ; /api/v1/projects/{project_id}/contents et /resolve ; /api/v1/folders ; /api/v1/files/{file_id}/move ou /copy ; GET /api/v1/jobs/{job_id}.

DONNÉES ET ÉTATS

File, Folder, Project, hiérarchie, intention durable de copie et statut temporaire de job.

DÉPENDANCES ET ÉCHANGES

IAM, PostgreSQL, S3, ARQ et statut Redis/Valkey.

COMPOTEMENT CIBLE PROPOSÉ

Files Rust garde des noms logiques et dossiers comme projection de navigation. Déplacer ou copier prépare une nouvelle édition FILESET ou une commande de présentation explicitement séparée. Un job expose un état durable et l'édition résultante.

MIGRATION DE CETTE CAPACITÉ

Importer toute la hiérarchie et les alias projet. Préserver conflits de noms et droits de destination.

Reconstituer les jobs encore ouverts avant bascule et éviter une copie physique inutile lorsque les blobs immuables peuvent être référencés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Copier vers un dossier interdit échoue même si la source est lisible. Une coupure après création de l'intention reprend sans doubler les entrées. Un autre utilisateur ne peut sonder le statut du job.

Références de l'analyse

[monorepo · services/files/src/file_management/api/v1/folders.py:264](#) Contrat de copie et destination.

[monorepo · services/files/src/file_management/api/v1/jobs.py:33](#) Propriété du job et traitement d'un statut absent.

[monorepo · services/files/src/file_management/api/v1/projects.py:97](#) Résolution du chemin de navigation.

FS-04 **Cible non qualifiée****Télécharger, prévisualiser et produire des archives****COMPOTEMENT OBSERVÉ**

Le téléchargement lit les octets S3 via le service. La garde de scan refuse INFECTED, PENDING et ERROR avec des réponses distinctes ; PENDING signale une attente temporaire. L'aperçu textuel a ses limites de type et de taille, mais ne bloque explicitement que INFECTED dans le code lu. Les dossiers et sélections se téléchargent en ZIP. Le pipeline ZIP borne les fichiers temporaires en vol, contrôle les dérives de taille et nettoie les ressources à l'arrêt du consommateur. Aucun presign n'est établi par les routes examinées.

UTILISATEURS ET RÔLES

Lecteurs autorisés et clients de téléchargement.

INTERFACES DANS LE CODE

GET /api/v1/files/{file_id}/download et /preview ; GET /api/v1/folders/{folder_id}/download ; POST /api/v1/projects/{project_id}/download.

DONNÉES ET ÉTATS

Blob, état de scan, réponse stream et fichiers temporaires ZIP.

DÉPENDANCES ET ÉCHANGES

IAM/gateway, S3 et pipeline ZIP.

COMPOTEMENT CIBLE PROPOSÉ

Un lecteur Rust résout une version de blob dans une édition approuvée puis applique IAM sur tous les chemins. Un EXPORT durable est un effet avec reçu et reprise. Un téléchargement interactif reste un stream médié avec contrôle des limites et révocation.

MIGRATION DE CETTE CAPACITÉ

Aligner les gardes preview, ZIP et download avant ouverture des corpus. Conserver CSV/texte et ZIP compatibles là où utilisés, puis ajouter l'identifiant d'édition dans le contrat de lecture.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un fichier en attente ou en erreur de scan n'est accessible ni par preview ni par ZIP. Annuler un stream libère les fichiers temporaires. Une lecture épinglée ne change pas de blob pendant le transfert.

Références de l'analyse

[monorepo · services/files/src/file_management/services/file_service.py:287](#) États bloquant le téléchargement.

[monorepo · services/files/src/file_management/services/file_service.py:1091](#) Garde spécifique et limites de preview.

[monorepo · services/files/src/file_management/services/zip_pipeline.py:9](#) Borne de ressources du pipeline ZIP.

FS-05 **Cible non qualifiée****Scanner, compresser et extraire du texte dans les workers****COMPORTEMENT OBSERVÉ**

ARQ exécute `scan_file_for_virus`, `compress_file`, `extract_file_text` et copie des objets d'un dossier. Les retries du scan sont bornés et les erreurs terminales sont persistées ; une tâche périodique reprend des scans PENDING restés sans résultat. L'extraction traite textes et PDF avec une limite de taille de fichier et de longueur de texte. La compression écrit une nouvelle clé puis fait pointer la ligne vers celle-ci avant de retirer l'ancien objet. Ces opérations enrichissent l'état de fichier actuel, sans créer une édition gouvernée.

UTILISATEURS ET RÔLES

Utilisateurs en attente de disponibilité ; exploitation des traitements.

INTERFACES DANS LE CODE

`WorkerSettings.functions` ; tâches `scan_file_for_virus`, `compress_file`, `extract_file_text` ; POST `/api/v1/projects/{project_id}/extract-text` selon le routeur projet.

DONNÉES ET ÉTATS

États antivirus, extraction textuelle, clé de blob compressé, jobs ARQ et intentions durables.

DÉPENDANCES ET ÉCHANGES

ARQ, antivirus, extraction PDF, stockage et base métier.

COMPORTEMENT CIBLE PROPOSÉ

Workers fongibles exécutant des runs bornés, avec résultat d'audit lié à une version immuable. Les transformations qui changent les octets créent un candidat et passent WAP ; l'audit post-candidat est lié durablement sans réécriture du summary par fast-forward.

MIGRATION DE CETTE CAPACITÉ

Conserver les résultats de scan avec leur hash/version. Identifier les transformations de compression déjà réalisées, réauditer les objets sans preuve suffisante et déplacer les tâches vers la file sans JVM retenue.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une panne d'antivirus finit en état explicite et ne publie pas le fichier. Une compression interrompue ne laisse aucune édition pointant vers des octets manquants. Un rejeu ne change pas une preuve d'audit déjà liée.

Références de l'analyse

[monorepo · services/files/src/file_management/workers/worker.py:83](#) Liste des tâches, bornes et reprise.

[monorepo · services/files/src/file_management/services/text_extraction_service.py:12](#) Limites de l'extraction.

[monorepo · services/files/src/file_management/workers/tasks.py:423](#) Parcours de compression asynchrone.

FS-06 **Cible non qualifiée****Rechercher noms et contenu sans élargir le périmètre****COMPORTEMENT OBSERVÉ**

La recherche distingue le chemin personnel du chemin de projet. `/my-files/search` fixe le propriétaire à l'appelant et ne reçoit pas de `projectId` permettant d'élargir la portée. `/projects/{id}/search` utilise le projet résolu et aucun filtre propriétaire, sous le contrôle `can_view` de la gateway. Le contrat accepte recherche

fulltext ou nom, type fichier/dossier, filtre dossier et pagination de 1 à 100 résultats par type. Fichiers et dossiers sont interrogés séquentiellement sur la session SQL partagée.

UTILISATEURS ET RÔLES

Utilisateurs recherchant leurs documents ou ceux d'un projet.

INTERFACES DANS LE CODE

GET /api/v1/my-files/search ; GET /api/v1/projects/{project_id}/search.

DONNÉES ET ÉTATS

Index PostgreSQL et texte extrait ; filtres sur projet, propriétaire et dossier.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, extraction et gateway IAM.

COMPORTEMENT CIBLE PROPOSÉ

Files et Catalog Rust indexent seulement les documents appartenant à des éditions approuvées. L'index est reconstructible ; snippets, compteurs et résultats appliquent la même politique que le téléchargement. Semantics/Embeddings reste une extension distincte de la recherche fulltext actuelle.

MIGRATION DE CETTE CAPACITÉ

Préserver chemins et filtres, puis reconstruire l'index depuis les éditions migrées. Ne pas reprendre le texte extrait d'un fichier supprimé ou d'un scan non accepté sans audit.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une recherche personnelle ne retourne pas les fichiers d'un autre utilisateur. Un projet interdit ne révèle ni texte ni compteur. Reconstruire l'index conserve les résultats autorisés de la même édition.

Références de l'analyse

[monorepo · services/files/src/file_management/api/v1/search.py:132](#) Portée personnelle fixée au sujet.

[monorepo · services/files/src/file_management/api/v1/search.py:165](#) Portée projet et gardes gateway.

FS-07 Cible non qualifiée

Restaurer la corbeille et différer la suppression physique

COMPORTEMENT OBSERVÉ

Les routes distinguent suppression logique, corbeille de projet, restauration de fichier ou dossier, suppression permanente et vidage. La destination de restauration est contrôlée. Pour supprimer définitivement un fichier, le service verrouille le namespace et la ressource, écrit le fait inverse d'autorisation, enregistre une intention durable de suppression S3 puis supprime la ligne. Les octets ne sont retirés qu'après commit, pour éviter une ligne conservée qui pointe vers un objet effacé.

UTILISATEURS ET RÔLES

Propriétaires, gestionnaires du projet et politique de conservation.

INTERFACES DANS LE CODE

/api/v1/trash/files/{file_id}/restore ; /api/v1/trash/folders/{folder_id}/restore ; DELETE /api/v1/trash/files/{file_id} ; POST /api/v1/projects/{project_id}/trash/empty.

DONNÉES ET ÉTATS

Tombstones logiques, parents de restauration, audit et intention durable de suppression des clés.

DÉPENDANCES ET ÉCHANGES

Outbox, IAM, stockage et futurs Maintenance/Audit.

COMPOTEMENT CIBLE PROPOSÉ

Maintenance Rust distingue retrait du service, conservation et purge physique. Les racines de GC viennent des éditions conservées, tags, runs et obligations de restauration. Les projections et refcounts restent reconstructibles ; un plan audité vérifie à nouveau les références avant effacement.

MIGRATION DE CETTE CAPACITÉ

Importer les éléments de corbeille et leurs délais, relier blobs aux éditions conservées et traiter les intentions anciennes. Inclure textes extraits et dérivés dans le plan de purge.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un rollback de transaction ne supprime aucun blob encore référencé. Une restauration respecte les droits du nouveau dossier. Une purge refuse un blob encore utilisé par une édition retenue, même si l'index de navigation est en retard.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/services/file_service.py:1000](#) Intention de suppression après commit et inverse authz.

[monorepo](#) · [services/files/src/file_management/api/v1/trash.py:62](#) Restauration et destination.

[monorepo](#) · [services/files/src/file_management/api/v1/trash.py:139](#) Vidage de la corbeille.

FS-08 **Cible non qualifiée****Servir les assets publics de branding dans un stockage distinct****COMPOTEMENT OBSERVÉ**

Les assets de configuration vivent dans une table dédiée, séparée des fichiers privés. Les routes d'administration requièrent une identité et la gateway impose le rôle global ; seule `/public/config/assets/{key}` sert sans authentification. Le service limite clés, MIME réel et taille, calcule un ETag et gère le cache conditionnel. Une mise à jour prend un verrou par clé, écrit un nouvel objet, commit la ligne puis nettoie l'ancien. Ces uploads d'administrateur n'ont pas de scan antivirus.

UTILISATEURS ET RÔLES

Administrateurs de branding et visiteurs de la page de connexion.

INTERFACES DANS LE CODE

GET `/api/v1/config/assets` ; PUT/DELETE `/api/v1/config/assets/{key}` ; GET `/api/v1/config/assets/{key}/info` ; GET `/api/v1/public/config/assets/{key}`.

DONNÉES ET ÉTATS

ConfigAsset, version de clé S3, hash/ETag et auteur de mise à jour.

DÉPENDANCES ET ÉCHANGES

Gateway IAM, S3 et caches HTTP.

COMPOTEMENT CIBLE PROPOSÉ

Files Rust conserve ce stockage public explicitement séparé, avec types et tailles admis. Ce parcours de configuration ne doit pas rendre public un FILESET métier ni contourner les politiques d'édition.

MIGRATION DE CETTE CAPACITÉ

Importer les assets et leur hash, conserver URLs et cache HTTP, puis changer l'implémentation derrière la même frontière gateway.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur ordinaire ne peut modifier un asset. Un MIME simulé est refusé avant S3. Deux uploads ou un upload concurrent à un delete ne laissent pas la ligne pointer sur un objet effacé.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/api/v1/config_assets.py:145](#) Route publique et réponse HTTP.

[monorepo](#) · [services/files/src/file_management/services/config_asset_service.py:120](#) Verrou, clé neuve et ordre commit/nettoyage.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Sessions d'upload, droits, dossiers, jobs, conservation et reçus d'effets restent des commandes/projections PostgreSQL.
- Une édition FILESET est un index Iceberg publié autoritaire, avec identité et version immuable de chaque blob.

Plan de données proposé

- Réception durable privée, calcul de hash, scan et extraction dans des runs bornés.
- Lecture médiée de blobs appartenant à l'édition autorisée ; pas de droit général au bucket ni aux fichiers staging.

Découpage du core Rust

- Sessions et navigation Files, ContentStore Rust, lecteur d'édition, outbox et adaptateurs d'objets.
- Writer et Publisher communs pour FILESET ; Maintenance pour GC et purge, indépendants du retard de navigation.

Identité, autorisation et isolation

- Uniformiser les gardes antivirus et publication sur download, preview, ZIP, extraction et indexation.
- Préserver le contrôle de destination sans remplacer un refus de dossier par un droit plus large. Les assets publics restent séparés.

Exploitation et reprise

- Surveiller scans PENDING, erreurs terminales, backlog ARQ/outbox, espace temporaire et intentions de suppression.

- Le protocole chunks actuel n'est pas qualifié pour reprise sur un autre pod. La restauration cible combine index Iceberg, blobs et commandes PG, sans considérer un index de recherche comme autorité.

Séquence de migration

- Inventorier IDs, dossiers, blobs, corbeille et jobs ouverts.
- Contrôler hash, immutabilité et audit avant adoption dans des éditions FILESET.
- Basculer les sessions et lecteurs puis les traitements vers Rust, un corpus à la fois avec un seul Publisher.
- Qualifier panne après publication avant ack, lecture de staging, reprise de chunks et GC sous retard de projection.

Écarts et décisions encore ouverts

- La différence de garde antivirus entre preview et download est observée statiquement ; son impact en déploiement n'a pas été testé.
- L'adoption sans copie exige maîtrise de la version physique et de la conservation du blob.
- Aucun scénario de charge, reprise, scan ou restauration n'a été exécuté dans ce lot.

Références de l'analyse

[monorepo](#) · [services/files/src/file_management/main.py:629](#) Assemblage des API Files.

[monorepo](#) · [services/files/src/file_management/workers/worker.py:80](#) Tâches et maintenance du worker.